ELLY Business Development Portal — CEO Requirements Gap Analysis

Date: 2026-10-31

Baseline: `PROJECT_AUDIT_2026-10-28.pdf` (original audit) and `PROJECT_AUDIT_2026-10-31.md/.pdf` (full re-audit completed earlier today, same branch). This document does not repeat either in full — it (1) summarizes the net delta against the 2026-10-28 baseline and (2) adds a dedicated classification of the 10 new CEO-facing requirements that were not explicitly scoped in either prior audit.

Scope verified against: `worktree-report-agent` branch, 11 commits ahead of `main`.

Method: Source-code verification only — every classification below cites the file(s) that back it. No code was modified. No functionality was assumed working from UI presence alone.

Part 1 — Delta Since the 2026-10-28 Baseline

This is a condensed restatement of the tracker already produced in today's full audit (`PROJECT_AUDIT_2026-10-31.md`, Section 11), included here so this document stands on its own.

#	2026-10-28 action item	Status today
1	~50% systemic pytest error rate	**Still open** — not re-measured with a full-suite run
2	No Alembic migrations	**Still open**
3	In-memory, single-process rate limiter	**Fixed (documented)** — constraint now stated in code
4	Duplicate `JWT_SECRET_KEY`/`TOKEN_ENCRYPTION_KEY` in `.env`	**Fixed**
5	No CI workflow	**Still open**
6	Nightly backup job gap	**Still open, worse** — latest backup file ~3.5 weeks stale
7	Missing tests (notifications/webhooks/KB/backups/signals)	**Fixed** (files added; pass/fail not independently re-verified)
8	Generated artifacts tracked in git	**Fixed**
9	Stale root-level `.db` files	**Fixed**
10	`NodeJS.Timeout` TypeScript error	**Fixed**
11	No webhook subscription frontend UI	**Still open**
12	Stale IHT port plan file	**Fixed**
13	JWT stored in `localStorage`	**Still open**
14	Agency-agents install decision	**Still open**

Net: 7 of 14 fixed, 7 still open. Of the original 6 HIGH-severity items, 2 are closed and 4 remain open.

No new regressions were found in the already-audited subsystems since the 2026-10-31 full audit ran a few hours ago on this same branch; the findings in that document (Connections board stage corruption, team-performance metric type mismatch, role-gate gaps, etc.) stand unchanged and are not repeated here.

Part 2 — New CEO Requirements: Classification

Each requirement below was checked against the actual backend route guards, service logic, and frontend wiring — not against the presence of a similarly-named page or button.

1. Separate executive account

Status: Partial.

A genuinely separate role exists: `DEV`, added this week specifically for executive-report access (`app/auth/deps.py`, enforced via `require_role("DEV")` in `app/routers/reports.py:18`). It is a real, distinct account type — not a relabeled admin account.

However, it does not behave as a coherent "executive" account:

- `DEV` cannot reach the admin console (`require_admin` excludes `DEV`), so an executive account cannot manage users, view audit logs, or change settings.
- `ADMIN` — the account that *can* do all of that — cannot generate or download the executive report (`require_role("DEV")` excludes `ADMIN`).
- There is no ownership check on report downloads: any `DEV` account can download any other `DEV` account's generated report by guessing a sequential report ID.

To close: decide whether `DEV` should be a superset of `ADMIN` (one executive account with full console + report access) or a genuinely separate role (in which case the admin console's user-management screens should surface DEV accounts, which they currently don't), and add an ownership check on report downloads.

2. Master admin console

Status: Partial.

A real admin console exists and is reasonably complete: dashboard, user list/create/detail, team view, showcase management, knowledge-base management, audit log viewer, and settings (`frontend/src/pages/admin/*`, gated by `AdminProtectedRoute` on the frontend and `require_admin` on the backend).

Defects that keep it from being a trustworthy "master" console:

- `GET /api/admin/users/{id}` throws an unhandled `AttributeError` and returns HTTP 500 (`admin.py:247` filters on `Meeting.organizer_id`, which doesn't exist on the model — the real column is `owner_id`). The admin user-detail page is broken for every user.
- The audit log — which should be the admin console's exclusive view into the system — is also readable by any BD/DEV user through a separate endpoint, undermining its "master console" exclusivity.
- No webhook subscription management lives in the console (see Part 1, item 11) despite webhooks being a backend-complete feature.

To close: fix the `organizer_id`/`owner_id` bug, restrict audit-log read access to admin-only, add a webhook management screen to the console.

3. Organization-wide employee monitoring

Status: Broken.

The CEO-facing Analytics page and team-performance views exist and are wired to real database queries (`app/services/report_agent_service.py`, `app/routers/analytics.py`). But the computation itself is wrong for most of what it reports: the per-rep activity counts for organizations added, contacts added, meetings completed, and follow-ups completed all compare an `Integer created_by` column against a name *string* — a type mismatch that means those four metrics silently read as zero for every rep (`report_agent_service.py:42-73`). Only communications and opportunities compute correctly. This feeds the Analytics page, the CSV export, and the new executive-report feature — a CEO looking at "employee monitoring" today sees most activity types understated to zero.

To close: this is the single highest-priority fix in this entire gap analysis — cast both sides of the comparison to the same type (compare `created_by` to the rep's `id`, not their name).

4. Role-based access

Status: Partial, with real security exposure.

Five roles exist (`BUSINESS_DEVELOPMENT`, `HEALTHCARE`, `ADMIN`, `USER`, `DEV`) and four distinct gate functions enforce them at the router level (`app/auth/deps.py`). This is a real, working RBAC layer for the routers that use it correctly.

The gap: several BD-only entity routers — Customers, Demos, Proposals, Negotiations, Tasks, Calls, Daily Priorities, and the Integrations ingest endpoint — only check `get_current_user` (i.e., "is logged in"), not `require_business_development`. A `HEALTHCARE` or `USER` account can read and modify all of this BD data through the API even though the frontend hides those pages from them. This is an active, not theoretical, access-control gap.

Additional gaps: no ownership checks on mutations (any BD user can delete any org or reassign any opportunity's owner with no audit trail on the reassignment); `owner_scope=all` is a client-supplied, unenforced filter rather than a real server-side scope restriction.

To close: add `require_business_development` to the 8 under-protected routers; add ownership checks to delete/reassign operations; enforce `owner_scope` server-side.

5. Private CEO workspace

Status: Missing.

No dedicated, CEO-exclusive workspace exists. Searching the frontend route tree (`frontend/src/App.tsx`) and page directory turns up no page scoped to a CEO-only audience distinct from the admin console — "CEO" only appears in the codebase as a job-title string in contact/organization forms, never as a page, route guard, or workspace concept. The closest things to it are:

- The Analytics page, which is **not** private — it's reachable by any BD or DEV user, not just ADMIN, so any rep can see every other rep's individual performance numbers.
- The admin console, which is private to ADMIN but is a management console, not a CEO-specific workspace (it has no CEO-specific views like cross-team pipeline health, risk flags, or a curated executive summary).

To close: this is a net-new feature, not a bug fix. It would need a new ADMIN-only (or new CEO-specific) route and page distinct from both the shared Analytics page and the general-purpose admin console, likely assembling: the (once-fixed) team-performance metrics, pipeline-wide KPIs, the executive report generator, and a restricted view of the audit log.

6. AI-powered business development automation

Status: Partial, with trust-breaking defects.

The Copilot is real: two-tier intent routing (regex fast-path + LLM function-calling fallback), live-traced and confirmed working, with genuine database-backed tools for search, discovery, drafting, and reporting

(app/services/copilot/*).

It falls short of "automation" in ways a CEO would care about:

- The Copilot's own stated policy is that it never autonomously sends email, WhatsApp, or LinkedIn messages (engine.py:205-209) — but one registered tool, send_funding_outreach, violates that policy by routing through CampaignService.quick_send, which auto-approves and sends with no human step.
- send_email doesn't send email at all — it writes a database row and replies "Successfully dispatched," which is a materially misleading response for anything relying on this as automation.
- There is no real workflow engine: TaskType.COMPLEX_WORKFLOW is handled as a single LLM tool-call loop, not an orchestrated multi-step workflow, and the regex fast-path can silently drop steps from a multi-part request (e.g., "find hospitals and draft emails" only performs the discovery half).
- update_organization_status is exposed to the LLM as callable but has no backing handler — it always errors.

To close: remove or properly gate send_funding_outreach behind the same confirm-step every other mutating tool uses; make send_email's response match what it actually does (or wire it to a real send path); either build a real workflow executor or stop describing COMPLEX_WORKFLOW as one; implement or remove update_organization_status.

7. WhatsApp assistant

Status: Missing (as an "assistant"); Partial (as a send tool).

What exists is a template-rendering send queue for a Chrome extension: app/routers/whatsapp_automation.py hands the extension a queue of leads with a merge-field-filled message template, and the extension drives web.whatsapp.com's own UI to send it and reports back success/failure. There is no AI involvement anywhere in this path — no LLM call, no reply handling, no conversation logic. It is accurately described as "WhatsApp send automation," not a "WhatsApp assistant." It also carries ToS risk since it automates WhatsApp Web's UI rather than using an approved API (same caveat as LinkedIn, below).

To close: if an actual assistant is wanted (AI-drafted messages per contact, inbound reply classification, conversational follow-up), that is new work — the current send-queue is not a foundation for it beyond the merge-field template system.

8. LinkedIn outreach

Status: Partial by design.

A working in-house Chrome extension (elly-outreach-extension/li-content.js) drives LinkedIn's own web UI to send connection requests and messages — this is real automation, not a stub. It has no inbound reply ingestion and no AI involvement, and the extension's own README documents that this approach breaches LinkedIn's Terms of Service and carries account-ban risk. This is an accepted, known trade-off already documented in-repo, not a new finding, but it bears repeating for the CEO-facing requirement list since the risk is operational (account bans), not just technical.

To close: no code fix closes this — it's a business decision about whether browser-automation risk is acceptable, or whether to move to LinkedIn's (paid, restrictive) official API.

9. Email integration

Status: Implemented.

This is the one fully real, production-grade integration in the list: Gmail OAuth, actual send, open/click tracking, reply sync, and bounce detection all work end-to-end (app/services/gmail_service.py and related routers), confirmed via the manual end-to-end test run completed earlier in this engagement (downloaded executive report referencing real seeded communications data). Two caveats carried over from the AI-automation finding above: the

click-tracking redirect is an unsigned open redirect with forgeable (sequentially-enumerable) tracking IDs, and this is the one channel where the Copilot's `send_email` tool claims to use it but doesn't actually call it.

To close: sign the click-tracking redirect; make the Copilot's `send_email` tool actually use this real, working integration instead of just logging a row.

10. Centralized workflow monitoring

Status: Missing.

No monitoring surface exists for the system's actual background work. There are 6 real APScheduler jobs running in-process (reply sync, deliverability checks, signal scans, backups, task reminders, etc.), but there is no dashboard, endpoint, or admin page that shows their run history, last-run time, success/failure state, or next scheduled run. A CEO asking "is outreach automation actually running" has no page to check — they would need someone to read application logs. This gap compounds the already-identified stale-backup problem (Part 1, item 6): had a monitoring view existed, that 3.5-week-old backup would have been visible days ago instead of being found only by this audit.

To close: net-new feature — a simple admin-console page listing each scheduled job, its last run time/result, and next run time, backed by a small table the scheduler writes to on each tick.

Part 3 — Updated Gap Summary

Capability	Status
Separate executive account	Partial
Master admin console	Partial
Org-wide employee monitoring	**Broken**
Role-based access	Partial
Private CEO workspace	**Missing**
AI-powered BD automation	Partial
WhatsApp assistant	**Missing**
LinkedIn outreach	Partial (by design, ToS risk)
Email integration	Implemented
Centralized workflow monitoring	**Missing**

Of the 10 new CEO requirements: **1 Implemented, 5 Partial, 3 Missing, 1 Broken.** None of the 10 is simply a UI-only illusion with no backend at all — even the weakest ones (WhatsApp, workflow monitoring) have real, if narrow, backend substance to build on. The most damaging single item is employee monitoring, because it is actively wrong rather than merely absent, and a CEO acting on those numbers today would be misled.

Part 4 — Prioritized Roadmap

P0 — Fix before the CEO relies on this data:

- Fix the `created_by` Integer-vs-string type mismatch in `report_agent_service.py` — this single fix corrects employee-monitoring, the Analytics page, and the executive report simultaneously.
- Fix `GET /api/admin/users/{id}` (`organizer_id` → `owner_id`).

- Add `require_business_development` to the 8 under-protected BD routers (Customers, Demos, Proposals, Negotiations, Tasks, Calls, Daily Priorities, Integrations ingest).
- Remove or confirm-gate `send_funding_outreach`; correct `send_email`'s response to match what it actually does.

P1 — Close the trust gaps in automation and access:

- Decide and implement the `DEV`/`ADMIN` relationship for executive accounts (superset vs. separate), and add ownership checks on report downloads.
- Restrict audit-log read access to admin-only; add ownership checks on org/opportunity mutations; enforce `owner_scope` server-side.
- Sign the Gmail click-tracking redirect.

P2 — Net-new CEO-facing features:

- Build a private CEO workspace (ADMIN-only view combining fixed KPIs, pipeline health, executive report access, and a scoped audit view).
- Build a centralized workflow/scheduler monitoring page.
- Decide whether a real WhatsApp/LinkedIn AI assistant (drafting, reply handling) is in scope, or whether the current send-queue automation is sufficient as-is.

P3 — Carried forward from the 2026-10-28 baseline, still open:

- Full-suite pytest re-run to confirm/close the ~50% error-rate claim.
- Add Alembic migrations.
- Add a CI workflow.
- Fix the stale-backup root cause.
- Build the webhook-subscription frontend UI.
- Decide on JWT storage hardening (localStorage vs. httpOnly cookie).
- Resolve the agency-agents install decision.

Compiled 2026-10-31 by source-code verification against `worktree-report-agent`. No code was modified in the course of this analysis.